Java para a OBI

Use esta apostila junto com os exercícios e com o guia da prova, que traz as regras e a estratégia. Ela tem só o que cai na OBI.

- **Seções 1 a 8:** a linguagem. Você precisa saber de cor.
- **Seções 9 a 13:** ferramentas que aparecem em quase toda prova.
- **Seções 14 a 19:** as técnicas das questões mais difíceis. Aprenda uma por vez, na ordem.
- **Seções 20 e 21:** leia antes de cada prova.

Nas tabelas, "custo" diz quantas operações o programa faz para uma entrada de tamanho N. O(N) é uma passada pelos dados. $O(N^2)$ é um laço dentro de outro. O(N log N) é o custo de ordenar. A seção 20 mostra qual custo cabe no tempo.

1. Estrutura de um programa

Salve este exemplo como `Main.java` para rodar localmente.

> **Na OBI:** o arquivo e a classe levam o nome da tarefa, em minúsculas. Tarefa com "Nome do arquivo: ogro.java" pede `public class ogro`. Veja o guia da prova.

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int a = entrada.nextInt();   // a = 3
        int b = entrada.nextInt();   // b = 8
        System.out.println(a + b);   // imprime 11
    }
}
```

Com a entrada `3 8`, a saída é `11`. Você também pode digitar um número por linha.

- `import` permite usar a classe `Scanner` pelo nome curto.
- `class Main` define a classe. Uma classe pública precisa ter o mesmo nome do arquivo.
- `main` é onde a execução começa. `void` indica que o método não devolve um valor.
- Chaves delimitam blocos. Declarações, atribuições e chamadas como `println` terminam com ponto e vírgula.
- `//` inicia um comentário de uma linha. `/* ... */` delimita um comentário de várias linhas.
- Java distingue maiúsculas de minúsculas. `String` e `string` são nomes diferentes.

```
javac Main.java
"3 8" | java Main
```

2. Como declarar e mudar valores

O padrão é `tipo nome = valor;`. Declare uma vez e use o nome para ler ou alterar o valor.

```
int pontos = 10;          // pontos = 10
pontos = pontos + 5;      // pontos agora vale 15
pontos += 2;              // 17
pontos++;                 // 18
final int LIMITE = 100;   // constante
```

Tipo	Exemplo	Uso
`int`	`int n = 42;`	Inteiros de -2^{31} até $2^{31}-1$
`long`	`long total = 3_000_000_000L;`	Inteiros de -2^{63} até $2^{63}-1$
`double`	`double media = 7.5;`	Valores com parte decimal
`boolean`	`boolean achou = false;`	Verdadeiro ou falso
`char`	`char letra = 'A';`	Um caractere simples, com aspas simples
`String`	`String nome = "Ana";`	Texto, com aspas duplas

Uma variável local precisa receber um valor antes de ser lida. Uma variável declarada dentro de um bloco só existe naquele bloco. O índice declarado em `for (int i = 0; ...)` fica disponível dentro do laço.

Operadores e conversões

Sintaxe	Significado
`+ - * / %`	Soma, diferença, produto, divisão e resto
`= += -= *= /=`	Atribuição e atualização
`== != < <= > >=`	Comparações que resultam em `boolean`
`&&`, `||`, `!`	E, ou, negação
`condicao ? a : b`	Escolhe `a` se a condição for verdadeira, ou `b` caso contrário

```
int quociente = 7 / 2;           // 3
double metade = 7 / 2.0;         // 3.5
int resto = 7 % 2;               // 1
int truncado = (int) 3.9;        // 3
int numero = Integer.parseInt("42"); // 42
long produto = 1L * 100_000 * 100_000; // 10.000.000.000
int maior = Math.max(12, 8);     // 12
```

Em um produto de dois `int`, a operação acontece em `int`, mesmo se o resultado for atribuído a um `long`. Converta um dos operandos antes de multiplicar. Para ordenar ou comparar inteiros, prefira `Integer.compare(a, b)` a `a - b`, que pode estourar.

3. Como pegar valores da entrada

Adicione `import java.util.Scanner;` no topo e crie um único leitor dentro de `main`.

```
Scanner entrada = new Scanner(System.in);
int idade = entrada.nextInt();        // 15
long populacao = entrada.nextLong();  // 3000000000
String palavra = entrada.next();      // "Ana"
char inicial = palavra.charAt(0);     // 'A'
```

A entrada `15 3000000000 Ana` fornece `15`, `3000000000`, `"Ana"` e `'A'`. Esses métodos leem tokens separados por espaços, tabulações ou quebras de linha.

Quero ler	Método
Inteiro	`nextInt()`
Inteiro grande	`nextLong()`
Decimal	`nextDouble()`
Uma palavra	`next()`
Uma linha inteira	`nextLine()`
Vários valores até acabar a entrada	`while (entrada.hasNextInt()) { ... }`

Para decimais com ponto, adicione `import java.util.Locale;` e chame `entrada.useLocale(Locale.US);` antes de ler.

Se o enunciado dá um inteiro sozinho na primeira linha e um texto na segunda, como `2` e `Ola mundo`:

```
int n = entrada.nextInt();          // 2
entrada.nextLine();                 // devolve "", o resto da primeira linha
String frase = entrada.nextLine(); // "Ola mundo", a segunda linha com espaços
```

Se o texto estiver na mesma linha do inteiro, a primeira chamada a `nextLine()` já devolve esse texto, incluindo o espaço separador.

Como imprimir

```
System.out.println(42);           // 42 e quebra de linha
System.out.print("3 ");           // sem quebra de linha
System.out.println(8);            // completa a linha "3 8"
System.out.printf(Locale.US, "%.2f%n", 3.5); // 3.50
```

Em uma submissão, imprima só o que o enunciado pede. Para muitas respostas, use `StringBuilder` e imprima uma vez ao final.

4. Condições e laços

```java
int nota = 7;
if (nota >= 7) {
    System.out.println("Aprovado");    // este ramo executa, pois 7 >= 7
} else if (nota >= 5) {
    System.out.println("Recuperacao");
} else {
    System.out.println("Reprovado");
}
```

```java
int soma = 0;
for (int i = 1; i <= 5; i++) { // início, condição, avanço
    soma += i;                 // soma: 1, 3, 6, 10, 15
}
// soma = 15

int faltam = 3;
while (faltam > 0) {
    faltam--;                  // faltam: 2, 1, 0
}
// faltam = 0

for (int i = 0; i < 10; i++) {
    if (i == 8) {
        break;             // sai do laço
    }
    if (i % 2 != 0) {
        continue;          // pula o restante desta iteração
    }
    System.out.println(i); // 0, 2, 4, 6, um por linha
}
```

Em `a && b`, Java só avalia `b` se `a` for verdadeiro. Isso permite escrever `i < v.length && v[i] == alvo` quando `i` é não negativo.

5. Vetores, matrizes e strings

```java
int[] v = new int[3];  // começa como {0, 0, 0}
v[0] = 9;              // v = {9, 0, 0}
int primeiro = v[0];   // 9
int ultimo = v[v.length - 1]; // 0; exige pelo menos um elemento

int[] dados = {9, -2, 4};
for (int i = 0; i < dados.length; i++) {
    System.out.println(dados[i]); // i = 0, 1, 2 imprime 9, -2, 4
}
for (int valor : dados) { // quando você só precisa do valor
```

```
    System.out.println(valor);    // valor = 9, -2, 4
}
```

Ler N números e guardar no vetor

```
int n = entrada.nextInt();        // 3
int[] valores = new int[n];       // [0, 0, 0]
for (int i = 0; i < n; i++) {
    valores[i] = entrada.nextInt(); // [8, 0, 0], [8, 4, 0], [8, 4, 9]
}
int primeiraPosicao = valores[0]; // 8; exige n > 0
int posicaoDoEnunciado = 2;
int valorNaPosicao = valores[posicaoDoEnunciado - 1]; // valores[1] = 4; exige n >= 2
```

Se a entrada for `3 8 4 9`, o vetor fica `[8, 4, 9]` e o valor da posição 2 do enunciado é `4`.

Para ler uma matriz com `linhas` e `colunas` já conhecidas:

```
int linhas = 2, colunas = 3;
int[][] grade = new int[linhas][colunas];
for (int i = 0; i < linhas; i++) {
    for (int j = 0; j < colunas; j++) {
        grade[i][j] = entrada.nextInt();
    }
}
// Com a entrada 1 2 3 4 5 6: grade = {{1, 2, 3}, {4, 5, 6}} e grade[1][0] = 4
```

```
String texto = "banana";
int tamanho = texto.length();     // 6
char letra = texto.charAt(0);      // 'b'
String pedaco = texto.substring(1, 4); // "ana", fim excluído
boolean igual = texto.equals("banana");  // true
boolean temAna = texto.contains("ana");  // true
int posicao = texto.indexOf('n');  // 2; devolve -1 se não encontrar
```

Use `v.length` para vetor, `texto.length()` para string e `lista.size()` para coleção. Compare o conteúdo de strings com `.equals()`. Para letras de `a` a `z`, `letra - 'a'` produz um índice de 0 a 25.

6. Métodos

Declare métodos dentro da classe, fora de `main`. Use `static` para chamar diretamente a partir do `main`.

```
static int contarPares(int[] valores) {
    int quantidade = 0;
    for (int valor : valores) {
        if (valor % 2 == 0) {
            quantidade++;
        }
```

```
    }
    return quantidade;
}
```

No `main`, chame com `int resposta = contarPares(new int[]{2, 7, 4});`. Dentro do método, `quantidade` passa por 1, 1 e 2, então `resposta` vale `2`. O tipo antes do nome é o tipo retornado. Um método `void` executa ações sem retornar um valor. Se um método alterar uma posição do vetor recebido, o chamador verá essa alteração.

7. Biblioteca que vale decorar

Adicione os imports correspondentes, como `import java.util.Arrays;`. Nas coleções genéricas, escreva `Integer` ou `Long` para números inteiros.

```
int[] v = {7, 2, 4};
Arrays.sort(v);                          // {2, 4, 7}
int indice = Arrays.binarySearch(v, 4);  // 1; negativo se ausente
Arrays.fill(v, -1);                      // {-1, -1, -1}

ArrayList<Integer> lista = new ArrayList<>();
lista.add(10);                           // [10]
lista.add(20);                           // [10, 20]
int item = lista.get(0);                 // 10
lista.set(0, 30);                        // [30, 20]
lista.sort(null);                        // [20, 30], ordem crescente

HashSet<Integer> vistos = new HashSet<>();
vistos.add(8);                           // {8}
boolean repetido = vistos.contains(8);   // true

HashMap<Integer, Integer> frequencia = new HashMap<>();
int valor = 8;
frequencia.put(valor, frequencia.getOrDefault(valor, 0) + 1); // {8=1}

ArrayDeque<Integer> fila = new ArrayDeque<>();
fila.addLast(10);                        // [10]
fila.addLast(20);                        // [10, 20]
int primeiro = fila.removeFirst();       // 10; fila = [20]

ArrayDeque<Integer> pilha = new ArrayDeque<>();
pilha.push(10);                          // [10]
pilha.push(20);                          // [20, 10], topo à esquerda
int topo = pilha.pop();                  // 20; pilha = [10]

PriorityQueue<Integer> menores = new PriorityQueue<>();
menores.add(20);
menores.add(10);
int menor = menores.remove();            // 10; resta 20
```

Use `isEmpty()` antes de remover se a coleção puder estar vazia. Para desempatar, faça a comparação explicitamente. `HashMap` e `HashSet` não garantem a ordem de iteração.

O que é "hash": é uma conta que transforma um valor em um número de gaveta. Para guardar o 8.345 em um armário de 100 gavetas, o Java calcula, por exemplo, o resto por 100 e usa a gaveta 45. Para saber se o 8.345 já apareceu, ele abre só a gaveta 45, sem olhar as outras. Por isso `contains`, `put` e `get` são rápidos mesmo com muitos valores. É a ideia do vetor de contadores `cont[x]++`, só que funciona com valores grandes, negativos e textos. Você não escreve essa conta, o Java já traz pronta.

Compare valores guardados em coleções com `.equals()` ou copie para um `int` antes. `lista.get(0) == lista.get(1)` compara objetos `Integer` e dá `false` para valores iguais acima de 127. Em uma `ArrayList<Integer>`, `lista.remove(0)` remove a posição 0, e `lista.remove(Integer.valueOf(0))` remove o valor 0.

8. Padrões de prova

Quando a questão pede...	Experimente...	Custo
Maior valor ou contagem	Percorrer uma vez, atualizando a resposta	O(N)
Frequência com valores de 0 a 100	`int[] cont = new int[101];` e `cont[x]++;`	O(N + 101)
Mediana	Ordenar e acessar `v[v.length / 2]` se N for ímpar	O(N log N)
Muitas somas em intervalos fixos	Pré-calcular prefixos	O(N + Q)
Descobrir se um valor já apareceu	Guardar valores em `HashSet`	O(N) em média
Ordenar por um critério e desempatar por outro	Comparador (seção 12)	O(N log N)
Conferir se o que abre também fecha	Pilha (seção 13)	O(N)
"O menor X tal que..." ou "o maior X tal que..."	Busca binária na resposta (seção 14)	O(N log X)
Maior trecho consecutivo que respeita um limite	Janela deslizante (seção 15)	O(N)
Menor número de passos em mapa ou rede	BFS (seções 16 e 17)	O(tamanho do mapa ou da rede)
Contar formas, ou melhor valor com escolhas que se repetem	Programação dinâmica (seção 18)	Depende da tabela
Escolha do momento que nunca prejudica o resto	Guloso, quase sempre após ordenar (seção 19)	O(N log N)

Com N = Q = 100.000, somar todos os valores de cada consulta pode chegar a 10 bilhões de operações. Para uma consulta com posições de 1 a N, use:

```java
long[] prefixo = new long[n + 1];  // n = 3: [0, 0, 0, 0]
for (int i = 1; i <= n; i++) {
    prefixo[i] = prefixo[i - 1] + entrada.nextInt(); // lê 2, -1, 4
}
// prefixo = [0, 2, 1, 5]
long soma = prefixo[r] - prefixo[l - 1]; // l = 2, r = 3: 5 - 2 = 3
```

`prefixo[0]` vale zero. Em `[2, -1, 4]`, os prefixos são `[0, 2, 1, 5]`. A soma de 2 até 3 é `5 - 2 = 3`. Veja o programa completo com leitura rápida.

9. Entrada grande

`Scanner` serve até cerca de 100.000 números. Acima disso, ele sozinho pode estourar o tempo. Use este modelo como ponto de partida de qualquer questão com entrada grande:

```java
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.util.StringTokenizer;

public class Main {
    static BufferedReader leitor = new BufferedReader(new InputStreamReader(System.in));
    static StringTokenizer tokens;

    // Devolve o próximo token, lendo outra linha quando a atual acaba.
    static String proximo() throws IOException {
        while (tokens == null || !tokens.hasMoreTokens()) {
            tokens = new StringTokenizer(leitor.readLine());
        }
        return tokens.nextToken();
    }

    static int proximoInt() throws IOException {
        return Integer.parseInt(proximo());
    }

    static long proximoLong() throws IOException {
        return Long.parseLong(proximo());
    }

    public static void main(String[] args) throws IOException {
        int n = proximoInt();                // 3
        long soma = 0;
        StringBuilder saida = new StringBuilder();
        for (int i = 0; i < n; i++) {
            soma += proximoInt();            // lê 5, 7, 9
            saida.append(soma).append('\n');  // 5, 12, 21
        }
        System.out.print(saida);             // uma única impressão
```

```
    }
}
```

- `throws IOException` é obrigatório em todo método que lê com `BufferedReader`.
- `proximo()` funciona com os valores na mesma linha ou em linhas separadas, como o `Scanner`.
- Milhares de `System.out.println` são lentos. Junte a saída em um `StringBuilder` e imprima uma vez.

Quando o enunciado não diz quantos valores vêm, leia até `readLine()` devolver `null`:

```
String linha;
int linhasLidas = 0;
while ((linha = leitor.readLine()) != null) {
    linhasLidas++;                    // processe a linha aqui
}
```

Com `Scanner`, o equivalente é `while (entrada.hasNext()) { ... }`. Não chame `proximo()` depois do fim da entrada: `readLine()` devolve `null` e o programa quebra.

Para imprimir valores na mesma linha, separados por espaço e sem espaço sobrando no fim:

```
int[] v = {4, 8, 15};
StringBuilder saida = new StringBuilder();
for (int i = 0; i < v.length; i++) {
    if (i > 0) {
        saida.append(' ');
    }
    saida.append(v[i]);
}
System.out.println(saida);            // "4 8 15"
```

10. Strings e letras

```
String s = "Obi2026";
char[] letras = s.toCharArray();                  // {'O','b','i','2','0','2','6'}
boolean digito = Character.isDigit(s.charAt(3));  // true
boolean letra = Character.isLetter(s.charAt(0));  // true
char minuscula = Character.toLowerCase('O');      // 'o'
int valor = s.charAt(3) - '0';                    // 2, o dígito como número
char terceira = (char) ('a' + 2);                 // 'c'
String maiusculas = s.toUpperCase();              // "OBI2026"
String texto = String.valueOf(42);                // "42"
String[] partes = "10 20 30".split(" ");          // {"10", "20", "30"}
int ordem = "ana".compareTo("bia");               // negativo: "ana" vem antes
String limpo = "  oi  ".trim();                   // "oi"
```

Um `char` é um número pequeno. Por isso `'a' + 2` precisa do `(char)` para voltar a ser letra, e `c - '0'` transforma o caractere de um dígito no valor dele. `compareTo` devolve negativo, zero ou positivo, na ordem do dicionário, com todas as maiúsculas antes das minúsculas.

Uma `String` não muda depois de criada. `s += x` copia o texto inteiro a cada vez, então montar um texto de N caracteres assim custa $O(N^2)$. Use `StringBuilder`:

```
StringBuilder sb = new StringBuilder();
sb.append("ab");                    // "ab"
sb.append('c').append(1);           // "abc1"
sb.reverse();                       // "1cba"
sb.setCharAt(0, 'x');               // "xcba"
sb.deleteCharAt(sb.length() - 1);   // "xcb"
String pronto = sb.toString();      // "xcb"
```

Contar letras com um vetor de 26 posições:

```
int[] cont = new int[26];
for (char c : "banana".toCharArray()) {
    cont[c - 'a']++;
}
// cont[0] = 3 (a), cont[1] = 1 (b), cont[13] = 2 (n)
```

Dois textos são anagramas se os vetores de contagem são iguais: `Arrays.equals(contA, contB)`.

Palíndromo com um índice em cada ponta:

```
static boolean ehPalindromo(String s) {
    int i = 0, j = s.length() - 1;
    while (i < j) {
        if (s.charAt(i) != s.charAt(j)) {
            return false;
        }
        i++;
        j--;
    }
    return true;
}
```

`ehPalindromo("arara")` devolve `true`, e `ehPalindromo("arame")` devolve `false`.

11. Números grandes e contas úteis

Tipo	Maior valor	Ordem de grandeza
`int`	`Integer.MAX_VALUE` = 2.147.483.647	$2 \cdot 10^9$
`long`	`Long.MAX_VALUE` = 9.223.372.036.854.775.807	$9 \cdot 10^{18}$

Regra prática: se os valores chegam a 10^9 e você soma vários ou multiplica dois, use `long`. Contadores e índices podem ficar em `int`.

```
int grande = Integer.MAX_VALUE;          // 2147483647
int estourou = grande + 1;               // -2147483648, sem aviso nenhum
```

```
long certo = (long) grande + 1;          // 2147483648
long absoluto = Math.abs(-7L);           // 7
int menor = Math.min(3, 9);              // 3
double raiz = Math.sqrt(49);             // 7.0
int teto = (7 + 2 - 1) / 2;              // 4: divisão de 7 por 2 arredondada para cima
int restoNegativo = -7 % 3;              // -1: o resto segue o sinal do dividendo
int restoPositivo = ((-7 % 3) + 3) % 3;  // 2
long arredondado = Math.round(2.5);      // 3
```

"Overflow" é quando a conta passa do maior valor do tipo e o resultado vira lixo, sem nenhum aviso. Para "infinito", use um valor que ainda pode ser somado sem estourar, como `Integer.MAX_VALUE / 2`. Para potências de inteiros, multiplique em um laço em vez de usar `Math.pow`.

Compare decimais com tolerância: `Math.abs(a - b) < 1e-9` em vez de `a == b`. Quando puder, fuja de `double`: comparar `a * d` com `c * b` em `long` substitui comparar `a / b` com `c / d`.

MDC, MMC e primos

```
static long mdc(long a, long b) {
    while (b != 0) {
        long resto = a % b;
        a = b;
        b = resto;
    }
    return a;
}

static long mmc(long a, long b) {
    return a / mdc(a, b) * b;   // divide antes de multiplicar para não estourar
}

static boolean ehPrimo(long n) {
    if (n < 2) {
        return false;
    }
    for (long d = 2; d * d <= n; d++) {   // basta testar até a raiz de n
        if (n % d == 0) {
            return false;
        }
    }
    return true;
}
```

`mdc(12, 18)` = 6, `mmc(4, 6)` = 12 e `ehPrimo(97)` = `true`. O MDC é o maior número que divide os dois, e o MMC é o menor múltiplo dos dois.

Dígitos de um número

```
int numero = 5082;
int somaDigitos = 0;
while (numero > 0) {
```

```
    somaDigitos += numero % 10;   // último dígito: 2, 8, 0, 5
    numero /= 10;                 // 508, 50, 5, 0
}
// somaDigitos = 15
```

12. Ordenação com critério

`Arrays.sort` em `int[]` só ordena em ordem crescente. Para ordem decrescente, ordene e percorra de trás para frente, ou use `Integer[]`:

```
Integer[] caixa = {7, 2, 4};
Arrays.sort(caixa, Collections.reverseOrder());  // {7, 4, 2}

int[][] pares = {{3, 1}, {1, 9}, {3, 0}};
Arrays.sort(pares, (a, b) -> {
    if (a[0] != b[0]) {
        return Integer.compare(a[0], b[0]);      // primeiro campo crescente
    }
    return Integer.compare(b[1], a[1]);          // no empate, segundo campo decrescente
});
// pares = {{1, 9}, {3, 1}, {3, 0}}

String[] nomes = {"bia", "Ana", "caio"};
Arrays.sort(nomes);                              // {"Ana", "bia", "caio"}
```

O comparador `(a, b) -> ...` recebe dois elementos e devolve negativo se `a` vem antes, positivo se `b` vem antes e zero se tanto faz. `Integer.compare(a, b)` dá ordem crescente. Trocar para `Integer.compare(b, a)` dá decrescente.

Quando cada item tem campos de tipos diferentes, crie uma classe pequena dentro da classe principal:

```
static class Aluno {
    String nome;
    int nota;

    Aluno(String nome, int nota) {
        this.nome = nome;
        this.nota = nota;
    }
}
```

```
ArrayList<Aluno> turma = new ArrayList<>();
turma.add(new Aluno("Bia", 8));
turma.add(new Aluno("Ana", 8));
turma.add(new Aluno("Caio", 10));
turma.sort((x, y) -> {
    if (x.nota != y.nota) {
        return Integer.compare(y.nota, x.nota);  // maior nota primeiro
    }
```

```
    return x.nome.compareTo(y.nome);            // no empate, ordem alfabética
});
// turma: Caio 10, Ana 8, Bia 8
```

Se a questão pede a posição original depois de ordenar, guarde o índice como mais um campo, por exemplo `{valor, indice}`.

13. Pilha na prática

Pilha resolve "o mais recente que ainda está aberto". O exemplo clássico é conferir parênteses:

```
static boolean balanceado(String s) {
    ArrayDeque<Character> pilha = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') {
            pilha.push(c);
        } else {
            if (pilha.isEmpty()) {
                return false;                   // fechou sem abrir
            }
            char abre = pilha.pop();
            if ((c == ')' && abre != '(') || (c == ']' && abre != '[') || (c == '}' && abre != '{')) {
                return false;                   // fechou o tipo errado
            }
        }
    }
    return pilha.isEmpty();                     // sobrou algum aberto?
}
```

`balanceado("([]{})")` devolve `true`. `balanceado("(]")` e `balanceado("((")` devolvem `false`. `pilha.peek()` olha o topo sem remover.

14. Busca binária

Em um vetor ordenado, cada comparação descarta metade do que falta. Decore esta versão, que acha a primeira posição com valor maior ou igual ao alvo:

```
// Primeiro índice i com v[i] >= alvo. Devolve v.length se não existir.
static int primeiroMaiorOuIgual(int[] v, int alvo) {
    int inicio = 0, fim = v.length;
    while (inicio < fim) {
        int meio = (inicio + fim) / 2;
        if (v[meio] >= alvo) {
            fim = meio;             // a resposta é meio ou está antes
        } else {
            inicio = meio + 1;      // a resposta está depois de meio
        }
    }
```

```
    return inicio;
}
```

Com `v = {2, 4, 4, 7}`: alvo 4 devolve 1, alvo 5 devolve 3 e alvo 9 devolve 4. A partir dela:

- O valor `x` existe se o índice `i` devolvido for menor que `v.length` e `v[i] == x`.
- A quantidade de valores entre `a` e `b`, inclusive, é `primeiroMaiorOuIgual(v, b + 1) - primeiroMaiorOuIgual(v, a)`.

Busca binária na resposta

Se a questão pede "o maior X que ainda funciona" e, quando um X funciona, todo X menor também funciona, não procure X diretamente. Escreva um método que testa um X e faça busca binária sobre X.

Exemplo: há cordas de vários comprimentos e você precisa de pelo menos `k` pedaços do mesmo tamanho. Qual é o maior tamanho possível?

```
static boolean consegue(int[] cordas, long tamanho, long k) {
    long pedacos = 0;
    for (int corda : cordas) {
        pedacos += corda / tamanho;
    }
    return pedacos >= k;
}

static long maiorPedaco(int[] cordas, long k) {
    long baixo = 0, alto = 1_000_000_000;   // a resposta está entre baixo e alto
    while (baixo < alto) {
        long meio = (baixo + alto + 1) / 2; // o "+ 1" evita laço infinito quando alto = baixo + 1
        if (consegue(cordas, meio, k)) {
            baixo = meio;                   // meio funciona, tente maior
        } else {
            alto = meio - 1;                // meio não funciona
        }
    }
    return baixo;                           // 0 significa que nem tamanho 1 funciona
}
```

`maiorPedaco(new int[]{8, 5, 7}, 4)` devolve 4: com tamanho 4 saem 2 + 1 + 1 = 4 pedaços, e com tamanho 5 saem só 3. Para "o menor X que funciona", troque para `meio = (baixo + alto) / 2`, `alto = meio` quando funciona e `baixo = meio + 1` quando não funciona.

15. Janela deslizante

Quando a resposta é um trecho consecutivo e todos os valores são positivos, mantenha uma janela de `inicio` até `fim`. O `fim` avança um por vez, e o `inicio` só avança quando a janela deixa de valer. Cada índice entra e sai uma vez, então o custo é O(N), não $O(N^2)$.

```
// Tamanho do maior trecho consecutivo com soma <= limite. Exige valores positivos.
static int maiorTrecho(int[] v, long limite) {
```

```
    int melhor = 0, inicio = 0;
    long soma = 0;
    for (int fim = 0; fim < v.length; fim++) {
        soma += v[fim];
        while (soma > limite) {
            soma -= v[inicio];
            inicio++;
        }
        melhor = Math.max(melhor, fim - inicio + 1);
    }
    return melhor;
}
```

`maiorTrecho(new int[]{2, 1, 3, 4, 1}, 6)` devolve 3, do trecho `2 1 3`.

16. Mapas em grade

Leia o mapa como uma matriz de caracteres, uma linha por vez:

```
int linhas = Integer.parseInt(leitor.readLine().trim());
char[][] mapa = new char[linhas][];
for (int i = 0; i < linhas; i++) {
    mapa[i] = leitor.readLine().toCharArray();
}
// mapa[i][j] é o caractere da linha i, coluna j
```

Para visitar os quatro vizinhos sem repetir código, use dois vetores de deslocamento. A busca em largura (BFS) abaixo calcula o menor número de passos, porque a fila visita primeiro tudo que está a 1 passo, depois a 2, e assim por diante.

```
static int[] dl = {-1, 1, 0, 0};   // cima, baixo, esquerda, direita
static int[] dc = {0, 0, -1, 1};

// Menor número de passos de (li, ci) até (lf, cf) sem pisar em '#'. Devolve -1 se não der.
static int menorCaminho(char[][] mapa, int li, int ci, int lf, int cf) {
    int linhas = mapa.length, colunas = mapa[0].length;
    int[][] dist = new int[linhas][colunas];
    for (int[] linha : dist) {
        Arrays.fill(linha, -1);            // -1 significa "ainda não visitado"
    }
    ArrayDeque<int[]> fila = new ArrayDeque<>();
    dist[li][ci] = 0;
    fila.addLast(new int[]{li, ci});
    while (!fila.isEmpty()) {
        int[] atual = fila.removeFirst();
        int l = atual[0], c = atual[1];
        for (int d = 0; d < 4; d++) {
            int nl = l + dl[d], nc = c + dc[d];
            if (nl < 0 || nl >= linhas || nc < 0 || nc >= colunas) {
                continue;                  // saiu do mapa
```

```
            }
            if (mapa[nl][nc] == '#' || dist[nl][nc] != -1) {
                continue;                  // parede ou já visitado
            }
            dist[nl][nc] = dist[l][c] + 1;
            fila.addLast(new int[]{nl, nc});
        }
    }
    return dist[lf][cf];
}
```

No mapa abaixo, ir de (0, 0) até (1, 2) leva 5 passos, contornando as paredes por baixo:

```
..#
.#.
...
```

Para contar regiões, como ilhas ou salas, percorra todas as células. A cada célula livre ainda não visitada, some 1 na resposta e rode a mesma BFS a partir dela para marcar a região inteira. Para oito vizinhos, use `dl = {-1, -1, -1, 0, 0, 1, 1, 1}` e `dc = {-1, 0, 1, -1, 1, -1, 0, 1}`.

17. Grafos

Um grafo tem vértices e arestas que ligam pares de vértices: cidades e estradas, pessoas e amizades. Guarde, para cada vértice, a lista dos vizinhos.

```
// Vértices numerados de 1 a n. Cada aresta é {a, b}.
static ArrayList<ArrayList<Integer>> montarGrafo(int n, int[][] arestas) {
    ArrayList<ArrayList<Integer>> vizinhos = new ArrayList<>();
    for (int i = 0; i <= n; i++) {
        vizinhos.add(new ArrayList<>());
    }
    for (int[] aresta : arestas) {
        vizinhos.get(aresta[0]).add(aresta[1]);
        vizinhos.get(aresta[1]).add(aresta[0]);   // apague esta linha se a aresta tiver mão única
    }
    return vizinhos;
}

// Menor número de arestas da origem até cada vértice. -1 marca quem não é alcançado.
static int[] distancias(ArrayList<ArrayList<Integer>> vizinhos, int origem) {
    int[] dist = new int[vizinhos.size()];
    Arrays.fill(dist, -1);
    ArrayDeque<Integer> fila = new ArrayDeque<>();
    dist[origem] = 0;
    fila.addLast(origem);
    while (!fila.isEmpty()) {
        int u = fila.removeFirst();
        for (int w : vizinhos.get(u)) {
            if (dist[w] == -1) {
```

```
                dist[w] = dist[u] + 1;
                fila.addLast(w);
            }
        }
    }
    return dist;
}
```

Com n = 5 e arestas 1–2, 2–3 e 1–4, as distâncias a partir de 1 são `[-1, 0, 1, 2, 1, -1]`. A posição 0 não é usada e o vértice 5 fica isolado. Para contar quantos grupos separados existem, rode a busca a partir de cada vértice ainda não visitado e conte quantas vezes ela começou.

18. Programação dinâmica

Use quando a resposta de um caso grande se calcula a partir de respostas de casos menores, e os mesmos casos menores aparecem muitas vezes. O roteiro é sempre o mesmo:

1. Diga em palavras o que `pd[i]` significa.
2. Escreva como `pd[i]` sai de posições anteriores.
3. Preencha os casos base e percorra na ordem em que as dependências já estão prontas.

Contar formas. De quantas formas dá para subir `n` degraus, subindo 1 ou 2 por vez? O último passo veio do degrau `i - 1` ou do `i - 2`.

```
static long formasDeSubir(int n) {
    long[] formas = new long[n + 2];
    formas[0] = 1;                          // ficar parado
    formas[1] = 1;
    for (int i = 2; i <= n; i++) {
        formas[i] = formas[i - 1] + formas[i - 2];
    }
    return formas[n];                       // n = 5: 1, 1, 2, 3, 5, 8
}
```

Maior soma de um trecho consecutivo. Em cada posição, o melhor trecho que termina ali ou continua o anterior ou recomeça.

```
static long maiorSoma(int[] v) {
    long melhor = v[0], atual = v[0];
    for (int i = 1; i < v.length; i++) {
        atual = Math.max(v[i], atual + v[i]);
        melhor = Math.max(melhor, atual);
    }
    return melhor;                          // {2, -5, 4, -1, 3} devolve 6, de 4 - 1 + 3
}
```

Troco com o mínimo de moedas. `minimo[t]` é o menor número de moedas que soma `t`.

```java
static int minimoMoedas(int[] moedas, int total) {
    int infinito = Integer.MAX_VALUE / 2;
    int[] minimo = new int[total + 1];
    Arrays.fill(minimo, infinito);
    minimo[0] = 0;
    for (int t = 1; t <= total; t++) {
        for (int moeda : moedas) {
            if (moeda <= t) {
                minimo[t] = Math.min(minimo[t], minimo[t - moeda] + 1);
            }
        }
    }
    return minimo[total] >= infinito ? -1 : minimo[total];
}
```

Com moedas `{1, 3, 4}` e total 6, a resposta é 2, com 3 + 3. Pegar sempre a maior moeda daria 4 + 1 + 1, três moedas. É por isso que este problema pede programação dinâmica, e não guloso.

19. Guloso

Um algoritmo guloso faz a melhor escolha do momento e nunca volta atrás. Ele só está certo quando essa escolha não atrapalha as seguintes, então teste a ideia em casos pequenos antes de confiar. Quase sempre o primeiro passo é ordenar.

Exemplo: dadas atividades com início e fim, quantas cabem sem sobreposição? Escolha sempre a que termina mais cedo, porque ela deixa o maior espaço livre para as outras.

```java
// Cada atividade é {inicio, fim}. Uma pode começar no instante em que a outra termina.
static int maximoAtividades(int[][] atividades) {
    Arrays.sort(atividades, (a, b) -> Integer.compare(a[1], b[1]));
    int quantidade = 0;
    int livre = Integer.MIN_VALUE;        // instante em que a última escolhida termina
    for (int[] atividade : atividades) {
        if (atividade[0] >= livre) {
            quantidade++;
            livre = atividade[1];
        }
    }
    return quantidade;
}
```

Com `{1, 3}`, `{2, 5}` e `{4, 6}`, a resposta é 2: a primeira e a terceira.

Se um caso pequeno feito à mão contradiz o guloso, como o troco da seção 18, a questão pede programação dinâmica.

20. Quanto cabe no tempo

Conte cerca de 100 milhões de operações simples por segundo. Olhe o maior N do enunciado antes de escolher a técnica:

Maior N	Custo que passa	Técnicas típicas
até 500	$O(N^3)$	Três laços aninhados
até 5.000	$O(N^2)$	Dois laços aninhados
até 1.000.000	O(N log N)	Ordenação, busca binária
até 10.000.000	O(N)	Uma passada, prefixos, janela deslizante
acima disso	O(log N) ou O(1)	Fórmula, busca binária na resposta

Um vetor de 10 milhões de `int` ocupa 40 MB, e de `long`, 80 MB. Uma matriz `int[5000][5000]` ocupa 100 MB. Confira o limite de memória antes de criar algo desse tamanho.

Em questões da OBI com pontuação parcial, uma solução simples e correta para os casos pequenos vale pontos. Escreva essa versão primeiro, submeta e só depois melhore.

21. Antes de submeter

1. Rode o exemplo da questão.
2. Crie um caso mínimo, um caso de empate e um caso no limite dos valores, se forem possíveis.
3. Confira se o enunciado numera as posições a partir de 1 e se o seu vetor começa em 0.
4. Estime o maior valor de uma soma ou produto antes de escolher entre `int` e `long`.
5. Confira nome da classe e do arquivo, formato da saída e ausência de mensagens de depuração.

Sintoma	O que verificar
`cannot find symbol`	Nome digitado, escopo e imports
`ArrayIndexOutOfBoundsException`	Se o índice está entre 0 e `length - 1`
`InputMismatchException`	Se o leitor espera o tipo correto e se um inteiro cabe no tipo
Linha vazia após `nextInt()`	Se `nextLine()` está lendo o fim da linha anterior
`NullPointerException`	Fim da entrada com `readLine()`, ou chave ausente em `mapa.get`
`NumberFormatException`	Espaço sobrando no texto passado a `parseInt`, ou número que não cabe em `int`
`StackOverflowError`	Método que chama a si mesmo sem parar
`ConcurrentModificationException`	Alteração de uma coleção dentro do `for` que a percorre
`unreported exception IOException`	Falta `throws IOException` no método que lê
Resposta errada	Empate, negativos, limites inclusivos, overflow e `==` entre `String` ou `Integer`
Resposta errada só em casos grandes	Overflow em soma ou produto, vetor pequeno demais
Tempo excedido	Custo da solução (seção 20), leitura com `Scanner`, muitas chamadas de impressão, `s += x` em laço
Memória excedida	Tamanho de vetores e matrizes (seção 20)

Volte aos exercícios e tente reescrever cada solução de memória depois de entender o gabarito.